

Military Asset Management System – Project Documentation

Developed By: Prabhav Rath

1. Project Overview

Introduction

The **Military Asset Management System** is a full-stack web application developed to help military commanders and logistics personnel efficiently manage and track military assets such as **weapons, vehicles, and ammunition** across multiple military bases.

Traditional inventory management methods, such as spreadsheets or manual records, can be difficult to maintain, prone to errors, and lack proper accountability. This application provides a centralized and secure platform where users can monitor inventory, record purchases, transfer equipment between bases, assign assets to personnel, record asset expenditures, and maintain a complete audit trail of every important transaction.

The system also implements **Role-Based Access Control (RBAC)** to ensure that users only have access to the features and data relevant to their responsibilities.

Objectives

- Digitally manage military assets across multiple bases.
- Maintain accurate inventory records.
- Track purchases, transfers, assignments, and expenditures.
- Provide complete transparency through audit logging.
- Restrict system access based on user roles.
- Present inventory information through an interactive dashboard.

Assumptions

- User accounts are created by the Administrator before system usage.
- Military bases and asset types are configured before normal users begin using the application.
- Inventory is maintained digitally without integration with physical tracking devices.
- Demo users and sample data are populated using the seed script.

Limitations

- The application is intended for logical inventory management only.
 - Physical GPS tracking or RFID-based asset tracking is not implemented.
 - Notifications and real-time messaging are outside the current project scope.
 - Barcode and QR code integration are not included.
-

2. Tech Stack & Architecture

Technology Stack

Layer	Technology
Frontend	React.js, Vite, Tailwind CSS
Backend	Node.js, Express.js
Database	MongoDB Atlas with Mongoose
Authentication	JSON Web Token (JWT)
Hosting	Vercel (Frontend), Render (Backend)

Why These Technologies Were Chosen

React.js

React was selected because it provides a component-based architecture that promotes reusable UI components, efficient rendering, and simplified state management.

Tailwind CSS

Tailwind CSS enables rapid UI development through utility-first classes, making it easy to build a responsive and modern interface without maintaining large CSS files.

Node.js & Express.js

Node.js with Express provides a lightweight and scalable backend capable of handling RESTful APIs efficiently. Express also simplifies middleware integration for authentication, authorization, validation, and logging.

MongoDB

MongoDB was chosen because it is a document-oriented database that works naturally with JavaScript objects. It allows flexible schemas and efficiently stores transaction-based data such as purchases, transfers, assignments, expenditures, and audit logs without requiring complex table joins.

JWT Authentication

JWT provides stateless authentication by securely transmitting user identity between the client and server. Each protected API validates the token before processing requests, eliminating the need for server-side session management.

High-Level Architecture

Frontend (React + Tailwind CSS)



REST API (Express.js)



Authentication & RBAC Middleware



Controllers



MongoDB Database (Mongoose)

3. Data Models / Database Schema

The application consists of the following MongoDB collections:

Users

Stores application users along with their authentication credentials, assigned role, and associated military base.

Fields include:

- Name
- Email
- Password (hashed)

- Role
 - Base ID
 - Created Date
-

Bases

Stores information about military bases.

Fields include:

- Base Name
 - Location
-

Assets

Contains the master list of military equipment.

Examples include:

- M16 Rifle
- AK-47
- Humvee
- Tank
- Ammunition

Each asset stores:

- Name
 - Equipment Type
 - Unit
 - Description
-

Inventory

Maintains the latest available stock of every asset at each military base.

Each inventory record contains:

- Asset
 - Base
 - Opening Balance
 - Current Stock
 - Last Updated
-

Purchases

Stores all purchase transactions.

Each record contains:

- Asset
 - Base
 - Quantity
 - Purchase Date
 - Supplier
 - Added By
-

Transfers

Stores asset movement between military bases.

Each transfer contains:

- Source Base
- Destination Base
- Asset
- Quantity
- Transfer Date
- Initiated By

Assignments

Stores records of assets assigned to military personnel or units.

Each assignment contains:

- Asset
- Base
- Quantity
- Assigned To
- Assignment Date

Expenditures

Stores assets that have been consumed, damaged, destroyed, or expired.

Each expenditure contains:

- Asset
- Base
- Quantity
- Reason
- Expenditure Date

Audit Logs

Maintains a complete history of all important system activities.

Each log contains:

- User
- Action
- Module
- Reference ID

- Previous Data
 - Updated Data
 - Timestamp
-

4. Role-Based Access Control (RBAC)

The system implements three different user roles to ensure secure access to data and operations.

Administrator

The Administrator has complete access to the application.

Permissions include:

- Manage users
 - Manage military bases
 - Manage asset types
 - View all dashboard data
 - Create purchases
 - Perform transfers
 - Create assignments
 - Record expenditures
 - View audit logs
-

Base Commander

The Base Commander manages operations only for the military base assigned to them.

Permissions include:

- View dashboard for assigned base
- Create purchases
- Transfer assets

- Assign equipment
- Record expenditures

Restrictions:

- Cannot access other military bases.
 - Cannot manage users.
 - Cannot view audit logs.
-

Logistics Officer

The Logistics Officer focuses only on logistics-related activities.

Permissions include:

- Create purchases
- Transfer assets
- View inventory

Restrictions:

- Cannot assign assets.
- Cannot record expenditures.
- Cannot manage users.
- Cannot view audit logs.

RBAC Implementation

Role-Based Access Control is enforced on both the frontend and backend.

On the frontend, navigation items and pages are shown or hidden based on the logged-in user's role.

On the backend, a custom **RBAC middleware** validates the user's role from the JWT token before allowing access to protected API endpoints. If a user attempts to access an unauthorized endpoint, the server returns a **403 Forbidden** response.

5. API Logging

To maintain transparency and accountability, the system records every important operation in the **Audit Logs** collection.

A reusable utility named **auditLogger.js** is responsible for generating audit log entries.

The following operations are logged:

- User Login
- Asset Purchases
- Asset Transfers
- Asset Assignments
- Asset Expenditures
- Asset Updates
- Asset Deletions

Each audit log stores:

- User ID
- Action
- Module
- Reference ID
- Previous Data (if applicable)
- Updated Data
- Timestamp

This provides a complete audit trail for all critical activities performed within the application.

6. Setup Instructions

Backend Setup

1. Clone the repository.
2. Navigate to the backend directory.

```
cd backend
```

3. Install dependencies.

```
npm install
```

4. Create a .env file and configure:

```
MONGO_URI=your_mongodb_connection_string
```

```
JWT_SECRET=your_secret_key
```

```
PORT=5000
```

5. Seed the database with demo data.

```
node utils/seed.js
```

6. Start the backend server.

```
npm run dev
```

Frontend Setup

1. Navigate to the frontend directory.

```
cd frontend
```

2. Install dependencies.

```
npm install
```

3. Create a .env file.

```
VITE_API_URL=http://localhost:5000/api
```

4. Start the frontend.

```
npm run dev
```

The application will now be available locally.

7. Key API Endpoints

Method	Endpoint	Description
POST	/api/auth/register	Register a new user
POST	/api/auth/login	Authenticate user and generate JWT
GET	/api/dashboard	Retrieve dashboard metrics
POST	/api/purchases	Create a purchase transaction
GET	/api/purchases	Retrieve purchase history
POST	/api/transfers	Transfer assets between bases
GET	/api/transfers	Retrieve transfer history
POST	/api/assignments	Assign assets to personnel
GET	/api/assignments	Retrieve assignment records
POST	/api/expenditures	Record asset expenditure
GET	/api/expenditures	Retrieve expenditure records
GET	/api/audit-logs	Retrieve audit logs (Admin Only)

8. Live Demo & Login Credentials

Deployment Links

Frontend (Vercel)

<https://military-asset-management-system-phi-one.vercel.app>

Backend (Render)

<https://military-asset-management-system-9dlr.onrender.com>

GitHub Repository

https://github.com/PrabhavRathi06/military_asset_management- system

Demo Login Credentials

Role	Email	Password
Administrator	admin@military.com	Admin@123
Base Commander	commander@military.com	Commander@123
Logistics Officer	logistics@military.com	Logistics@123

Conclusion

The Military Asset Management System provides a secure, scalable, and user-friendly solution for managing military assets across multiple bases. By combining inventory management, transaction tracking, dashboard analytics, role-based access control, and audit logging, the application ensures transparency, accountability, and efficient asset management.

The project demonstrates the practical implementation of full-stack web development using the MERN ecosystem while following modern software engineering practices such as modular architecture, secure authentication, authorization, RESTful APIs, and responsive user interface design.